PROJECT GUIDE

Islet

A Dynamic-Island-style notch for Windows 11

Music, Claude Code and Codex activity, notifications, battery and more,
in one small black pill at the top of your screen.

MADE BY	VERSION	PLATFORM	BUILT WITH
Gopinath	**2.0.0**	**Windows 10 / 11**	**Electron 37**

© 2026 Gopinath. All rights reserved.

CONTENTS

Inside this guide

01	Why Islet exists	12	Build, test, debug
02	Every feature	13	Privacy, troubleshooting, uninstall
03	How it looks: every state	14	What's new in 2.0
04	Tech stack	15	Islet in plain English
05	Architecture	16	Words you need
06	How each part works	17	Follow the data
07	Repository map	18	Read the code yourself
08	Install and run	19	Engineering ideas it shows
09	Using Islet	20	Hard problems and limits
10	Claude Code and Codex	21	Interview questions and answers
11	Settings	22	Cheat sheet and practice

8	20	13	148
data sources	settings	notch states	passing tests

© 2026 Gopinath. All rights reserved.

SECTION 01

Why Islet exists

Phones and Macs keep live status in one place at the top of the screen. Windows spreads it across the taskbar, the tray, a corner of banners and hidden terminal windows. Islet pulls it into one pill.

One glance

See what is playing, what your AI coding agent is doing, and new messages without switching windows.

Act in place

Skip songs, seek, change volume, and Allow or Deny Claude Code permission requests from the notch.

Out of the way

Click-through except over the notch, hides in fullscreen apps, and goes to a single dot when idle.

Local and light

No account, no cloud, no tracking, and no unsigned helper exe. Runs from source with Smart App Control on.

The real thing



Open notch: agent sessions on the left, Spotify in the middle, time and battery on the right, tools at the edge. The real notch page with sample data.

© 2026 Gopinath. All rights reserved.

SECTION 02

Every feature

Grouped by area. Everything below ships in this repository today.

Music

- Spotify, or any player using Windows media controls (YouTube in a browser, VLC, ...)
- Album art, title, artist; accent colour taken from the art; art clears when a track has none
- Previous / play-pause / next; click the bar to seek
- Scroll over the music side to change Spotify's own volume
- Like the song (Spotify's Alt+Shift+B shortcut)
- Live wave from Spotify's audio only (24 FFT bands); stops drawing once the bars settle
- Timed lyrics (lrclib.net) under the seek bar
- Enlarged album view over a blurred backdrop
- Song-change peek; optional Spotify Web API (Premium)

Claude Code and Codex

- Sessions from both agents in one column, newest first (up to five; waiting sessions are always kept)
- Status per session: working, needs you, done
- Last 3 actions (tools, edited files, replies; your prompts for Codex)
- Allow / Deny Claude Code permission requests from the notch
- Session chips to switch project; Codex chips have a blue dot
- Hooks install on first run, with a backup; unticking removes only Islet's handlers
- Falls back to reading transcripts when hooks are off

Notifications

- Every Windows toast except Spotify's becomes a notch pop-up NEW with the full message: 400 px wide, grows up to about 7 lines
- Display time grows with message length (4.5 s + 40 ms per character, max 12 s)
- Real app logos; click to open the app; hover to keep it
- Windows' own banners off while running, restored on quit NEW even after a crash
- NEW On the lock screen Windows' banners come back, so messages still show there
- NEW Hidden pop-ups wait, then show (latest 5)
- Do Not Disturb, with an optional daily schedule

Battery and system

- Battery level and charging state
- NEW Black hole animation when the charger goes in (~2.8 s, "CHARGING INITIATED") or out (~1.8 s, "RUNNING ON BATTERY"); Cinematic or Subtle style
- Charging: pulsing green edge glow and a streak on the bottom edge (paused while the black hole plays)
- One-time low-battery warning
- Mic, camera and screen-capture dots while in use
- Clock and date column; hides behind fullscreen apps

Info

- Weather by city (Open-Meteo, no key), C or F
- Next calendar event from a private ICS link; cancelled events, EXDATEs and moved occurrences handled NEW follows each event's time zone
- Browser download progress and a finished pop-up
- Timer from the tray, shown as a chip

Tools and window

- Clipboard history: last 5 texts, memory only
- File shelf: drop files in, drag them out (no duplicates)
- Hover to open, click elsewhere to close
- Ctrl+Alt+N: pill, open, hidden; Ctrl+Alt+M: next monitor
- NEW Alt+F4 can't close the notch by mistake; use tray > Quit
- Settings window, tray menu, start with Windows, single instance

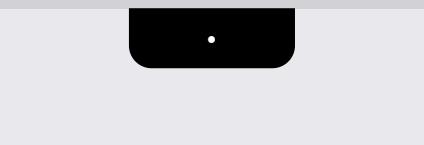
© 2026 Gopinath. All rights reserved.

SECTION 03

How it looks: every state

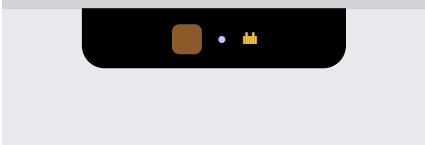
The notch is always black and hangs from the top edge of the screen. It changes size and content with what is going on.

Closed pill



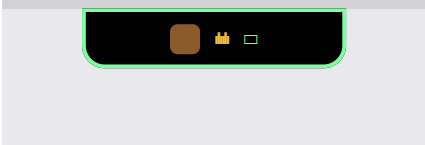
Quiet

Nothing playing, not charging, agent idle: one dot.



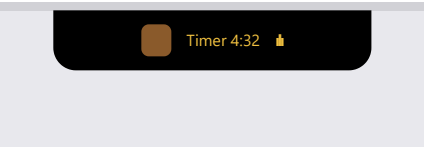
Playing

Album art, Claude dot (lit when active), live wave.



Charging

Green edge glow pulses; a light streak runs along the bottom.



Timer / meeting / download

A chip shows the most urgent item.



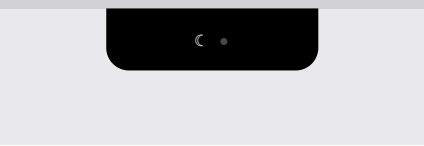
Agent needs you

Amber pulsing dot while a permission waits.



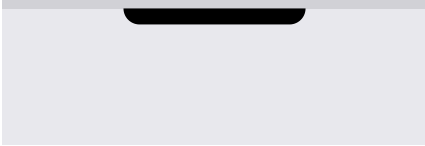
Mic / camera / screen in use

Orange, green and red dots after the art.



Do Not Disturb

Moon icon; notification pop-ups aren't shown.



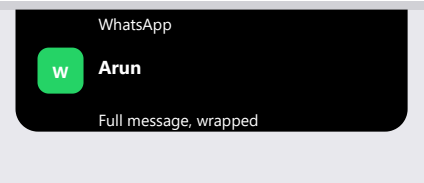
Idle line

Optional (idleHide): a thin 7 px line.

Hidden

Ctrl+Alt+N or a fullscreen app: nothing on screen.

Pop-ups and open



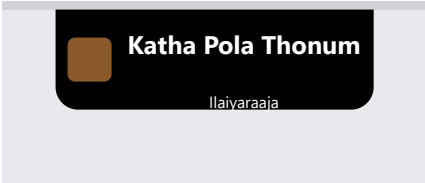
Notification pop-up NEW

400 px wide, height grows with the text.



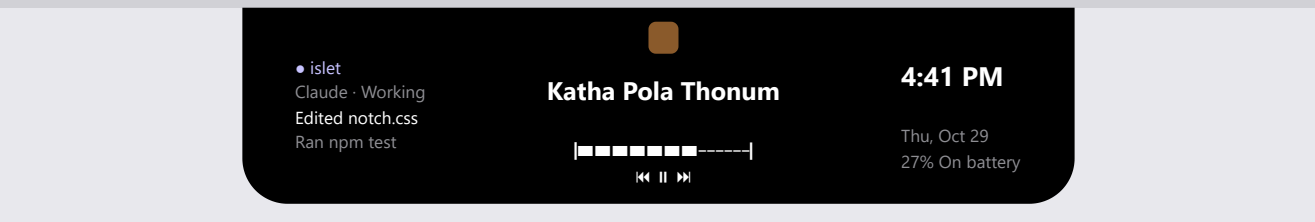
Black hole NEW

340 x 84 px, plays on charger in / out.



Song change peek

Opens the full notch for 2.5 s on a new song.



Open

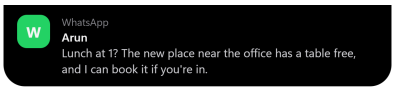
Hover: 640 x 150 px. Agents, music, time and battery, tools. Click elsewhere to close. The green glow follows the edge while charging.

© 2026 Gopinath. All rights reserved.

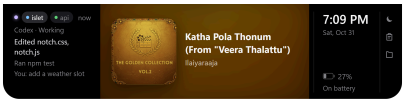
Real screenshots



Pill



Pop-up



Album view

The real notch page with sample data.

SECTION 04

Tech stack

Small and native to Windows. No UI framework, no database server, no cloud.

Layer	Technology	Used for
App shell	Electron 37 (Chromium + Node.js)	Frameless, transparent, always-on-top window; tray; global hotkeys; powerMonitor; IPC
UI	HTML, CSS, vanilla JavaScript	The notch (renderer/) and the settings window (settings/)
Animation	Canvas 2D	Black hole charger animation (renderer/blackhole.js); runs only while playing
Main process	Node.js built into Electron (CommonJS)	Window control, config, timers, HTTP bridge, spawning helpers
Windows access	Windows PowerShell 5.1	Media session, privacy dots, fullscreen detection
Native code	C# compiled at runtime (Add-Type)	Spotify process-loopback audio capture, FFT, volume
Storage	node:sqlite (read-only), JSON	Notification database; %APPDATA%\islet\config.json
Registry	HKCU Notifications\Settings	Turning Windows' banners off and back on
Network	fetch()	lrclib.net, Open-Meteo, your ICS link, optional Spotify Web API
Agent hooks	HTTP server on 127.0.0.1:47321	Claude Code and Codex hooks post events
Tests	node --test	18 files, 148 tests
Packaging	electron-builder 25 (NSIS)	One-click per-user installer

© 2026 Gopinath. All rights reserved.

Key decisions

Decision	Why
No UI framework	One small view. Vanilla JS starts fast and is easy to read.
C# inside PowerShell	Smart App Control blocks unsigned exes. Add-Type compiles inside signed powershell.exe.
Read the notification DB	Windows has no notification-listener API for unpackaged apps.
Hooks over loopback HTTP	Instant agent status; transcript timing is the fallback.
powerMonitor for the charger	Plug / unplug events arrive instantly, no polling.
Windows media controls	No Spotify login or Premium needed; works with other players too.

© 2026 Gopinath. All rights reserved.

SECTION 05

Architecture

Electron's two-process model. The main process talks to Windows and the web through helpers; the notch page only sees a small, safe API.

Layer	Parts
Windows + web	Media session (SMTC) · Spotify audio (process loopback) · Registry (consent, banners) · Notification DB (wpndatabase.db) · Web APIs (lyrics, weather, ICS) · Downloads folder · Claude / Codex hooks · Power events
Helpers	media.ps1 · spotify-audio.ps1 + .cs · system.ps1 · notifications.js + icons.js · lyrics.js, weather.js, calendar.js, spotify-web.js · downloads.js · shelf-file.js · claude-bridge.js + hooks/islet-hook.js
Main process	main.js: window · tray · hotkeys · powerMonitor · config.json · IPC · starts every helper
Bridge	preload.js (window.bridge) · settings/preload.js (settings:load / save)
Screen	renderer/ (index.html · notch.css · notch.js · blackhole.js) · Settings window

How data moves

- **In:** PowerShell helpers print lines, hooks POST, and the .js helpers call back. main.js decodes helper output with one UTF-8 decoder per stream (the hook and the HTTP bridge do the same), parses it and forwards with webContents.send.
- **Draw:** notch.js keeps the latest state and calls render() on every event and every 500 ms.
- **Out:** clicks and scrolls call window.bridge (ipcRenderer.send). main.js runs them: starts media.ps1 with the command, writes to spotify-audio stdin, or answers the waiting hook request.
- **Mouse:** the window is click-through. Entering the notch sends hover(true); main.js makes it clickable until the pointer leaves.
- **Lifetime:** the notch window only closes when the app quits (tray Quit or Windows shutdown). Alt+F4 is ignored so helpers never talk to a destroyed window.

© 2026 Gopinath. All rights reserved.

SECTION 06

How each part works

Music	media.ps1 reads the Windows media session once a second and prints JSON; toggle, next, prev, seek and like each start media.ps1 once with that command. spotify-audio captures only the Spotify process tree into 24 log-spaced bands about 30 times a second. lyrics.js caches timed lyrics per song and resends the current ones when the page reloads. Spotify Web API answers that a newer request has overtaken are dropped, so Like acts on the right track.
Agents	claude-bridge.js runs an HTTP server on 127.0.0.1:47321 and keeps a status per session. hooks/islet-hook.js POSTs each event. A PermissionRequest is held for permissionWaitSeconds; with no answer the terminal asks as usual.
Notifications	notifications.js polls wpndatabase.db read-only every 2 s and parses the toast XML. Banners are hidden with ShowBanner = 0; old values are written to banners.json before the registry changes and kept until the restore is verified. On lock they are put back; on unlock hidden again.
Battery	main.js forwards powerMonitor on-ac / on-battery / resume. The renderer ignores the first report, dedupes, debounces 200 ms, and plays blackhole.js; an opposite event cancels it. Reduced motion shows a still frame.
System	system.ps1 reads the CapabilityAccessManager consent store for mic, camera and screen use, and detects fullscreen windows.
Info	weather.js (every 15 min). calendar.js: next timed event in 24 h; jumps straight to the current repeat, skips cancelled and excluded ones. Times follow TZID, including Outlook's Windows zone names; repeats keep their clock time in the event's own zone. DAILY and WEEKLY rules with INTERVAL, COUNT, UNTIL, BYDAY and WKST are followed; a rule with anything else, like BYMONTH, shows only its first date. downloads.js: .crdownload / .part / .partial / .download, each finished file announced once.
Settings	Own window and preload. main.js validates every value, writes config.json via a temp file, and applies hand edits within 2 s. A broken config.json is left alone, not overwritten. A save's answer only refills fields you haven't changed since, so typing during an autosave is kept.

© 2026 Gopinath. All rights reserved.

SECTION 07

Repository map

Path	Lines	Purpose
main.js	653	Main process: window, tray, hotkeys, power events, config, IPC, starts every helper
preload.js	44	window.bridge: the only API the notch page can call
renderer/index.html	111	Notch markup: pill, pop-up, open view, panels
renderer/notch.css	291	Notch styling, states and animations
renderer/notch.js	572	Open / close, music, agents, battery, pop-ups, shelf, render loop
renderer/blackhole.js	224	Canvas black hole for charger in / out
settings/	177	Settings window: HTML, CSS, JS and its own preload
claude-bridge.js	192	Hook HTTP server; installs and removes Claude Code / Codex hooks
hooks/islet-hook.js	52	Run by each agent hook; POSTs the event
media.ps1	121	Windows media session reader and controller
spotify-audio.ps1 / .cs	325	Spotify loopback capture, FFT bands, volume
system.ps1	71	Mic / camera / screen dots, fullscreen detection
notifications.js	174	Notification database reader, banner toggle
icons.js	114	Real app logos
lyrics.js / weather.js	68 / 47	lrclib.net lyrics, Open-Meteo weather
calendar.js	209	Next event from an ICS link, time zones and repeats
downloads.js	77	Browser download progress
shelf-file.js	23	Reads and checks shelf.json
spotify-web.js	122	Optional Spotify Web API (PKCE login)
test/	1316	18 test files run by node --test
demo.html	191	Original browser design demo (not used by the app)
docs/ · package.json	- · 54	Screenshots · scripts, Electron version, installer config

© 2026 Gopinath. All rights reserved.

SECTION 08

Install and run

You need Windows 10 1903+ or 11 and Windows PowerShell 5.1 (part of Windows). Running from source also needs Node.js 20+ with npm, and Git.

Option A: download the installer

Get `Islet-Setup-<version>.exe` from `github.com/goprocker/Islet/releases/latest` and run it. It installs to %LOCALAPPDATA%\Programs\islet, adds Islet to the Start menu and starts it. Islet doesn't update itself, so to update, install the new version. Quit any `npm start` copy first, as only one Islet runs at a time.

The installer is unsigned. SmartScreen may warn about it, and Smart App Control may block it. If it's blocked, use Option B.

Option B: run from source

```
git clone https://github.com/goprocker/Islet.git
cd Islet
npm install
npm start
```

Code changes apply on the next restart. Good for development.

Build your own installer

```
npm run dist      # creates dist\Islet Setup <version>.exe (run from PowerShell, not Git Bash)
```

Then run the setup exe. The installed copy does not follow code edits; rebuild and reinstall to update.

On first run, Islet adds itself to Start with Windows and installs the agent hooks.

© 2026 Gopinath. All rights reserved.

SECTION 09

Using Islet

Action	How
Open	Hover the notch
Close	Move away, or click anywhere else
Cycle pill / open / hidden	Ctrl+Alt+N
Next monitor	Ctrl+Alt+M, or tray > Show on screen
Volume / seek	Scroll over the music side / click the progress bar
Album view	Click the album art
Like	Heart button, shown for Spotify (sends Spotify's Alt+Shift+B)
Clipboard / shelf / DND	Buttons in the tools column
Timer · Settings	Tray > Timer · Tray > Settings...
Quit	Tray > Quit (Alt+F4 does nothing on the notch)

SECTION 10

Claude Code and Codex

Islet installs the hooks on first run. Tray > Connect to Claude Code and Codex removes or adds them. Adding saves a backup next to each file.

Hook event	What the notch shows
UserPromptSubmit, PostToolUse	Working, with the latest actions
PermissionRequest, permission Notification	Needs you; Claude Code requests get Allow / Deny
Stop, any other Notification	Done
SessionStart (Codex) · SessionEnd	New session · session leaves the list

- Hooks run with node, so Node.js must be on your PATH, even with the installed app.
- Hooks point at the copy that installed them. After moving it or switching, untick and tick the option.
- Claude Code: ~/.claude/settings.json (backup settings.json.islet-backup).
- Codex: ~/.codex/hooks.json if ~/.codex exists (backup hooks.json.islet-backup). Approve Islet's hooks once in Codex (/hooks).
- Removing hooks deletes only Islet's handlers, even when they share a matcher group with your own.
- permissionWaitSeconds (default 8) is how long a request waits for you in the notch. 0 turns holding off.
- Codex permission requests show 'Needs you' but are answered in Codex.

© 2026 Gopinath. All rights reserved.

SECTION 11

Settings

Stored in %APPDATA%\islet\config.json and edited from the settings window.

Key	Default	Meaning
size	normal	compact, normal or large
accent	album	Colour from album art, or #rrggbb
idleHide	false	Thin line when nothing is happening
powerAnim NEW	true	Black hole animation when the charger goes in or out
powerAnimStyle NEW	cinematic	cinematic, or subtle (no particles or wave, softer glow, slower spin)
hideInFullscreen	true	Hide behind fullscreen apps
notifications	true	Show Windows notifications in the notch
hideWindowsBanners	true	Turn off Windows' own banners while running (back on at the lock screen)
dnd / dndFrom / dndTo	off	Do Not Disturb, optional daily HH:MM window
permissionWaitSeconds	8	Allow / Deny hold time, 0 to 24
lyrics	true	Fetch lyrics from lrclib.net
weatherCity / weatherUnit	empty / C	City name (empty turns weather off); C or F
calendarIcsUrl	empty	Private iCal link
downloadsFolder	empty	Empty means Downloads
clipboardHistory	true	Keep the last 5 copied texts in memory
displayId	null	Screen to show on (set by Ctrl+Alt+M or the tray)
spotifyClientId	empty	Optional Spotify Web API (Premium)

If config.json can't be read (for example during a hand edit), Islet keeps the current settings and leaves the file as it is.

SECTION 12

Build, test, debug

```
npm test        # node --test test/*.test.js  (18 files, 148 tests)
npm run dist    # NSIS installer, run from PowerShell

# inspect the notch with Chrome DevTools
node_modules\electron\dist\electron.exe . --remote-debugging-port=9333
```

Open chrome://inspect to see the page. Handy globals: setOpen(true), render().

© 2026 Gopinath. All rights reserved.

SECTION 13

Privacy, troubleshooting, uninstall

Privacy

- Everything runs on your PC. Network calls only go to lrclib.net, Open-Meteo, your ICS link, and Spotify if connected.
- Clipboard history never touches disk. The shelf stores file paths only.
- The hook server listens on 127.0.0.1 only. The notch page has no direct Node access.
- Registry changes stay under HKCU: banner flags, put back on quit, and the Start with Windows entry.

Troubleshooting

Problem	Fix
Banners still bottom-right	Start Islet normally (sandboxed shells can't write the registry). Sign out and in once if needed.
Banners off after a crash	Start Islet and quit from the tray, or turn the setting off.
Codex column empty	Approve Islet's hooks in Codex (/hooks).
npm run dist fails in Git Bash	Run it from PowerShell.
Installer blocked	Smart App Control; run from source (section 08, Option B).
Agent status stops updating	Check that node is on your PATH. If Islet moved, untick and tick Connect to Claude Code and Codex.
Installed app ignores my code changes	Rebuild with npm run dist and run the new setup exe.
Second launch does nothing	Only one copy runs; it opens the settings of the running one.
Notch won't close with Alt+F4	By design; use tray > Quit.
Calendar event missing	Its time zone isn't recognised (skipped with a console warning). A repeat rule with parts like BYMONTH shows only its first date.
Shelf starts empty	shelf.json wasn't a valid list. The original is kept as shelf.json.invalid.

Uninstall

- Tray: untick Connect to Claude Code and Codex, and Start with Windows.
- Tray: Quit (puts Windows banners back).
- Installed app: Windows Settings > Apps > Islet > Uninstall. From source: delete the folder.
- Delete %APPDATA%\islet.

© 2026 Gopinath. All rights reserved.

SECTION 14

What's new in 2.0

Version 2.0 closes 14 reliability issues and collects everything added since the first guide. Nothing changes in how you use Islet, and no settings were added or renamed.

Fixed in 2.0

- **Calendar**: events follow their own time zone instead of the PC's; repeats keep their time across daylight saving; rules Islet doesn't fully understand no longer invent dates, so they show only their first date.
- **Notifications**: the banner recovery record is saved before banners are turned off; CDATA text displays correctly; pop-ups that arrive while hidden wait and resume instead of being lost.
- **Music**: a late Spotify reply can't make Like act on the wrong track; lyric lines with several timestamps show at every one; lyrics return after a page reload; a slow album cover can't override a fixed accent colour.
- **Settings**: typing during an autosave is kept, and a change made before the settings load no longer saves an empty form.
- **Shelf and downloads**: a damaged shelf.json no longer causes a startup error and is kept as shelf.json.invalid; downloads finishing together each announce their own file.
- **Agents**: emoji and other non-English text in prompts, folders and session names stay intact.
- **Tests**: 8 files, 43 passing → 18 files, 148 passing.

Also since the first guide

- **Published release**: a ready-made installer on GitHub Releases, so you no longer have to build it yourself.
- **Player controls**: previous, next, play and pause redrawn with balanced shapes.
- **Black hole charger animation** with Cinematic / Subtle styles and an on/off switch (replaces the plain charging pop-ups).
- **Full notification text**: wider, taller pop-ups that stay longer for long messages; Windows banners return on the lock screen.
- **Alt+F4 fix**: the notch no longer closes and throws errors; quit from the tray.
- **Reliability fixes**: config.json never overwritten when broken, safe banner recovery, UTF-8 helper output, Calendar exceptions, shelf duplicates, waiting agent sessions kept, hook removal limited to Islet's own, wave loop stops when idle, charger glow vs. black hole / hide / idle conflicts.

© 2026 Gopinath. All rights reserved.

SECTION 15 · LEARN THE PROJECT

Islet in plain English

From here on, the guide teaches the project from zero, so you can explain it yourself.

The one-sentence version

> Islet is a small always-on-top Windows app. Some background helpers watch your PC, one manager program collects what they report, and a tiny web page draws it as a black pill at the top of the screen.

Picture a small office

In the office	In the code	Its job
The manager	`main.js` (main process)	The only one with the keys. Can read files, change the registry and start programs. Starts everyone else and passes messages along.
The shop window	`renderer/` (the notch page)	An ordinary web page: HTML, CSS, JavaScript. Draws what it is told. Can't touch your files or Windows by itself.
The service hatch	`preload.js`	A small opening between the two, with a fixed menu of allowed requests (`window.bridge`). Nothing else gets through.
Scouts	`media.ps1`, `system.ps1`, `spotify-audio`, `notifications.js`…	Each watches one thing, such as the song or the mic, and reports it to the manager.
The doorbell	`claude-bridge.js`, `hooks/islet-hook.js`	Claude Code and Codex ring it when something happens, so Islet doesn't have to keep checking.
The rule book	`config.json`	Your settings. The manager reads it and re-reads it when it changes.

The three directions data moves

- **In:** a scout or doorbell tells the manager something ("the song changed").
- **Across:** the manager passes it through the hatch, and the page redraws.
- **Back:** you click Next. The page asks through the hatch, and the manager tells Windows to skip.

Nearly every feature is this same loop. **Learn it once and you understand most of the 3,700 lines.**

Why it is split up like this

- **Safety:** a web page that can only ask from a fixed menu can't harm your PC, even if a song title or message contains hostile text.
- **Crash isolation:** if a scout dies, the manager restarts it 2 seconds later and the notch keeps running.
- **Right tool for each job:** web tech draws; PowerShell and C# reach what JavaScript can't.

© 2026 Gopinath. All rights reserved.

SECTION 16

Words you need

Every term an interviewer may use, in one line each.

Term	What it means here
Electron	Chromium (to draw) plus Node.js (to reach the system), packed together, so a web page can become a desktop app.
Main / renderer	Electron's two sides: main is the Node.js manager, renderer is the web page in a window.
Preload, contextBridge	A script that runs before the page and hands it a safe, hand-picked API. The page gets `window.bridge`, not Node.js.
IPC	Inter-process communication: messages between processes. `ipcRenderer.send` and `webContents.send` are the two directions.
stdin, stdout	A program's input and output text streams. Helpers print one JSON line to stdout; main reads it. Volume commands go in via stdin.
Polling vs events	Polling keeps asking "anything new?". Events get told. Islet polls the notification database and uses events for charger changes.
Hook	A command a tool runs at a set moment. Claude Code runs Islet's script when it stops, uses a tool or needs permission.
127.0.0.1, port	127.0.0.1 means "this same PC", so traffic never leaves the machine. A port (47321) is a numbered door on it.
SMTC	Windows' media controls: the service that knows what is playing, in Spotify, a browser or VLC, and can skip or seek it.
Process loopback	A Windows feature that captures the sound of one app only. That's why the wave ignores other apps.
FFT	Fast Fourier Transform: maths that turns sound samples into "how loud is each pitch". Those are the wave's bars.
Registry	Windows' settings database. Islet uses it to switch banners off and to see mic, camera and screen use.
wpndatabase.db	The SQLite file where Windows stores notifications. Islet opens it read-only.
requestAnimationFrame	Browser call meaning "run this before the next screen draw". Used for smooth animation.
Debounce	Wait for things to go quiet before reacting, so a flaky charger doesn't fire many times.
Race condition	A bug that depends on which of two things finishes first, such as an old reply arriving after a new one.
Atomic write	Write to a temp file, then rename. The real file is always the whole old version or the whole new one.

© 2026 Gopinath. All rights reserved.

SECTION 17

Follow the data

Being able to trace one action end to end is what convinces an interviewer you know a project.

A new song starts

1. Spotify tells Windows what it is playing. `media.ps1`, already running, asks Windows once a second.
2. `Get-Player` chooses: Spotify if playing, else any playing app, else paused Spotify, else whatever Windows picks.
3. It prints one line of JSON: title, artist, position, duration, cover as base64 text, and source app.
4. `startMedia()` in `main.js` rebuilds whole lines with `lines()`, parses them and calls `send("media", d)`.
5. It also asks `lyrics.js` for the words. `preload.js` delivers the media message to `onMedia` in `notch.js`.
6. `notch.js` stores the track, picks the accent colour from the cover, opens for 2.5 s because the title changed, and calls `render()`.
7. Windows reports position only now and then, so `render()` estimates it: last position + time since. It runs every 500 ms.

You click Next

1. `window.bridge.control("next")` sends an IPC message from the page.
2. The `control` handler in `main.js` checks the word is toggle, next or prev, and starts `media.ps1 next`.
3. That copy calls Windows' `TrySkipNextAsync` and exits. The watcher reports the new song, and the first story repeats.

A WhatsApp message arrives

1. Windows saves the toast in `wpndatabase.db`. Every 2 s `notifications.js` asks for rows newer than the last ID seen.
2. On its first look it only records the newest ID, so old messages don't all pop up at launch.
3. It pulls the text out of the toast's XML and skips Spotify, because that's already shown as music.
4. `main.js` drops it if notifications are off or Do Not Disturb is on, works out what a click should open, and finds the real logo.
5. `notch.js` queues it. Pop-ups show one at a time for 4.5 s + 40 ms per character, 12 s at most; hovering keeps one open.

The wave moves

1. The C# helper captures only Spotify's sound and keeps the latest 1,024 samples (about 23 ms) in a ring buffer.
2. About 30 times a second, FFT turns them into 24 bands from 40 Hz to 16 kHz, each scored 0 to 255.
3. `main.js` forwards only the newest set every 33 ms, and `frame()` eases each bar 35% toward it, so motion looks smooth.

© 2026 Gopinath. All rights reserved.

Claude asks permission: the most impressive flow

1. Claude Code wants to run a command, so it starts `node islet-hook.js PermissionRequest` and gives it the details on stdin.
2. The hook POSTs JSON to `127.0.0.1:47321/claude` and **waits for the reply**.
3. `claude-bridge.js` validates the event, marks the session "waiting", starts a timer and **keeps the HTTP response open**.
4. `onPermission` reaches the page. `showPerm()` shows the card and holds the notch open.
5. You click Allow. `answerPerm` sends it through the hatch to `bridge.answer`, and `settle` finally replies: `{"decision":"allow"}`.
6. The hook prints the decision in the format Claude Code expects and exits. Claude runs the command.

> **If nobody clicks:** after `permissionWaitSeconds` (default 8) the reply says `null`, the hook prints nothing, and Claude asks in the terminal as usual. **If Islet isn't running:** the connection fails, the hook exits quietly, and again Claude just asks. Islet can never block your work.

Three time limits nest so the shortest wins cleanly: bridge at most **24 s**, hook gives up at **25 s**, Claude Code kills the hook at **30 s**.

You plug in the charger

1. `powerMonitor` fires `on-ac`, an instant Windows event with no polling. `main.js` sends `power`.
2. `notch.js` waits 200 ms for it to settle. The first report at startup only records the state, so nothing plays at launch.
3. `BlackHole.play` draws on a canvas for about 2.8 s, then its animation loop stops and costs nothing.

How a window can be see-through to clicks

The window is 672 × 190 px, transparent and always on top, though you only see the black notch inside it. Normally it ignores the mouse (`setIgnoreMouseEvents(true, {forward:true})`), so clicks pass to the app below while Chromium still sees the pointer move. Entering the notch sends `hover(true)`, which makes it clickable. It opens after 120 ms and closes 400 ms after you leave, so a passing pointer causes no flicker.

What happens at startup

1. Single-instance lock: if a copy already runs, this one exits and the running one opens Settings.
2. Load `config.json`, create the window, then start the hook server (installing hooks on first run) and the tray.
3. Spawn the three PowerShell helpers (`media.ps1`, `spotify-audio.ps1`, `system.ps1`).
4. Register power events, timers, downloads, the shelf, banners, the notification watcher, the config watcher and hotkeys.
5. When the page finishes loading, send it everything it needs to catch up.

© 2026 Gopinath. All rights reserved.

SECTION 18

Read the code yourself

Don't start at line 1 of everything. This order takes about an hour, and each file builds on the last.

#	Read	Look for
1	`preload.js` (44 lines)	The complete list of what the page can do. It works as a table of contents.
2	`app.whenReady()` at the bottom of `main.js`	The whole app's startup on one screen.
3	`startMedia()` in `main.js`, then `media.ps1`	The simplest full helper loop.
4	`render()` in `renderer/notch.js`	How state becomes what you see.
5	`hooks/islet-hook.js`, then `claude-bridge.js`	The Allow / Deny flow from the last page.
6	`test/lyrics-follow.test.js`	How the code is tested without a real network.

Four snippets worth knowing by heart

1. Rebuilding lines from a stream (`main.js`). Data arrives in random-sized chunks, so a line or even an emoji can be split in two. The decoder holds back half a character, and `pop()` keeps the unfinished line for next time.

```
const decoder = new StringDecoder("utf8");
let buf = "";
stream.on("data", d => {
  buf += decoder.write(d);
  const parts = buf.split("\n");
  buf = parts.pop();
  for (const l of parts) if (l.trim()) fn(l.trim());
});
```

2. The hatch (`preload.js`). The page gets named functions, never `ipcRenderer` itself, so it can't send arbitrary messages.

```
contextBridge.exposeInMainWorld("bridge", {
  control: cmd => ipcRenderer.send("control", cmd),
  onMedia: on("media"),  // ...and the rest of the fixed menu
});
```

3. A save that can't corrupt (`main.js`). If power fails during the first line, the real file is untouched. Rename swaps it in one step.

```
const tmp = configPath() + ".tmp";
fs.writeFileSync(tmp, JSON.stringify(config, null, 2));
fs.renameSync(tmp, configPath());
```

4. Ignoring a reply that came too late (`calendar.js`, `weather.js`). Each new request gets a number. A slow reply from an old request sees a different number and is dropped.

```
const gen = ++generation;
// ...later, when the download finishes:
if (gen === generation && text !== null) onEvent(nextEvent(text));
```


© 2026 Gopinath. All rights reserved.

SECTION 19

Engineering ideas it shows

Interviewers care less about the notch than about these ideas. Each one is real code you can point to.

Idea	In plain words	Where in Islet
Least privilege	Give each part only the power it needs.	The page only gets `window.bridge`. Windows helpers stay outside it.
Validate at the boundary	Trust nothing that comes from outside.	`validEvent` rejects bad hook events; `settings:save` checks every value; `control` allows 3 words.
Safe rendering	Outside text never becomes HTML.	Titles and messages use `textContent`; link schemes are allowlisted.
Atomic write	Never leave half a file.	`saveConfig`, `saveJournal`.
Write-ahead journal	Record how to undo before you change.	Old banner values reach `banners.json` before the registry changes, so a crash can be undone.
Fail open	If the helper fails, the main task carries on.	The hook exits quietly, and Claude asks in its terminal.
Fallback	Keep a second, weaker source.	Hooks give exact status; transcript timing is the guess when they're off.
Self-healing	Restart what dies.	Each helper restarts 2 s after it exits.
Debounce	Wait for it to go quiet.	Charger events: 200 ms.
Hysteresis	Need more evidence to change state.	Fullscreen needs 2 equal readings. Low battery warns at 20%, re-arms above 25%.
Dropping stale work	An old answer mustn't overwrite a new one.	`generation` counters, `accentRequest`, `asked ===` key in lyrics.
Backpressure	Don't send faster than it can be used.	Wave keeps only the newest levels, 30 times a second at most.
Bounded memory	Every list has a limit.	5 clips, 12 shelf files, 50 click targets, 5 hidden pop-ups, 1 MB hook body.
Idle costs nothing	Stop work when nothing changes.	The wave and black hole animation loops stop themselves.
Don't damage user files	Change only your part, and keep a backup.	Hook install backs up first and removes only Islet's handlers; a broken file is refused.
Testable design	Pass dependencies in, so tests can fake them.	`follower(send, fetchLines)` in `lyrics.js`.

© 2026 Gopinath. All rights reserved.

SECTION 20

Hard problems and limits

"Tell me about a hard bug" is likely. Tell it as: problem, cause, fix, lesson.

Problem	Cause	Fix	Lesson
Helper exe blocked by Windows	Smart App Control blocks unsigned programs.	Compile the C# at runtime with `Add-Type` inside the signed PowerShell.	Platform limits shape design.
Allow / Deny card vanished instantly	Node fires `close` on the request once its body is read, so it looked as if the hook had left.	Watch `res.on("close")` with `!res.writableEnded`.	Check what an event really means.
Terminal prompt delayed 25 s	Claude holds its prompt while the hook waits.	Default hold cut to 8 s, set by a setting.	A feature can harm the flow around it.
Banners stayed off after a crash	The old values were only safe once written.	Write the journal first; restore on the next run, and delete an entry only once verified.	Plan for a crash at every step.
Emoji turned to garbage	Multi-byte characters were split across chunks.	One `StringDecoder` per stream.	Streams are bytes, not text.
Alt+F4 caused errors	It destroyed the window while helpers kept sending to it.	Refuse `close` unless really quitting.	Think about lifetimes.
Like hit the wrong track	A late Spotify reply described the previous song.	Drop replies that a newer request has overtaken.	Async order isn't guaranteed.
Album art was 0 bytes	PowerShell 5 couldn't pick the right overload on a COM object.	Call `AsStreamForRead` through reflection.	Know your runtime's limits.

Known limits: say these before you're asked

- **The hook port has no password.** Any program on the PC, and possibly a web page, could POST a fake status or a fake permission card. It can't answer your real prompts, but a shared secret token would be a good next fix.
- **The notification database is undocumented.** A Windows update could change it. It was chosen because the official listener needs a packaged app.
- **Polling means small delays:** up to 2 s for a notification and about 1 s for music.
- **Electron is heavy:** the installer is about 94 MB, for one small pill. Each music command also starts a new PowerShell.
- **Tests cover the logic modules, not the screen or real Windows calls.** `main.js` needs Electron to run, so it has no direct tests, and mouse actions are checked by hand.
- **Hooks need Node.js on PATH,** even with the installed app. The installer is unsigned, and there are no automatic updates.

What I would do next

Add a token to the hook server. Split `main.js` into smaller modules. Keep one PowerShell open for media commands. Add end-to-end UI tests. Get a code-signing certificate and add auto-update.

© 2026 Gopinath. All rights reserved.

SECTION 21

Interview questions and answers

Understand these, then say them in your own words. Don't memorise them.

Your pitch

> **30 seconds:** "Islet brings the iPhone's Dynamic Island idea to Windows. It's an Electron app that shows music, notifications, battery and AI coding agent status in one pill at the top of the screen. You can skip songs or approve a Claude Code permission request from it. The interesting part is reaching Windows features from JavaScript using PowerShell and C# helpers, while keeping the web page locked down."

About the project

What problem does it solve?

Windows spreads live status across the taskbar, tray, banners and terminals. Islet puts it in one place, and you can act on it without switching windows.

Walk me through the architecture.

Three layers. PowerShell, C# and Node modules collect data. `main.js` owns the window, settings and helpers. A sandboxed web page draws. Helpers talk to main through stdout lines, and main talks to the page through IPC with a preload bridge.

Why Electron rather than a native app?

HTML and CSS make a transparent, heavily animated shape easy, and Node gives me processes, files and HTTP. The cost is size and memory: about a 94 MB installer. For this kind of UI I accepted that.

Why vanilla JavaScript with no React?

It's one small view that updates from events. A framework would add a build step and weight for little gain. `render()` simply sets the screen from state.

What was the hardest part?

Choose Allow / Deny. One HTTP request stays open while a person decides, across three time limits and three ways to end: notch click, terminal answer, or nothing. It must never block Claude.

© 2026 Gopinath. All rights reserved.

Electron and processes

Main process vs renderer?

Main is Node.js, there's one of it, and it has full system access. A renderer is a Chromium page per window. Islet has two: the notch and settings.

Why a preload script?

Giving a page Node.js means any injected script could run commands. With context isolation, preload exposes only named functions, so the page has a fixed menu.

How does the window let clicks through?

`setIgnoreMouseEvents(true, {forward:true})` passes clicks to the app below but still reports mouse movement. On `mouseenter` the page tells main to turn it off.

Why spawn helpers instead of doing it all in Node?

Node can't call Windows media or audio APIs directly. PowerShell can use WinRT and compile C#, needs no install, and a helper crash can't take down the app.

What if a helper crashes?

Each has an `exit` handler that restarts it after 2 s unless the app is quitting. The delay stops a crash loop from hammering the CPU.

Why `app.exit(0)` and not `app.quit()` for a second copy?

`quit()` is asynchronous, so the rest of the script could start helpers first. `exit` stops it immediately.

Windows and audio

How do you read notifications with no public API?

The listener API needs a packaged app identity. So I open Windows' `wpndatabase.db` read-only with `node:sqlite` every 2 s and take rows above the last ID. It works but is undocumented, so it's a known risk.

How does the wave show only Spotify?

Windows process loopback captures one process tree. The C# helper finds the oldest Spotify process and captures it. Then: Hann window, FFT, 24 log-spaced bands, and values 0 to 255 printed as text.

Why log-spaced bands?

We hear pitch by ratios, so each octave feels equal. Equal-width bands would give almost all the bars to high notes.

Why C# compiled at runtime?

Smart App Control blocks unsigned exes. `Add-Type` compiles in memory inside `powershell.exe`, which is signed.

How do you know the camera is in use?

Windows keeps `LastUsedTimeStart` and `LastUsedTimeStop` per app under CapabilityAccessManager in the registry. Stop = 0 means still in use.

Why not the Spotify Web API?

It needs a Premium developer account, and it's Spotify-only. Windows media controls need no login and work for other players. The Web API remains optional, for like state and up next.

© 2026 Gopinath. All rights reserved.

Reliability and async

What race conditions did you meet?

Slow replies overwriting new state: old calendar links, old weather cities, an album cover overriding a fixed colour, and a Spotify reply acting on the previous track. Each request gets a number or key, and stale replies are dropped.

How do you stop a crash leaving the user's PC changed?

Banners. Old values go to `banners.json` atomically before the registry changes. On the next start they can be restored, and an entry is only removed once the registry confirms it.

What if `config.json` is broken?

Keep the current settings and leave the file alone, because it may be a hand edit in progress. Only a missing file gets created.

How do you edit a user's `settings.json` safely?

Read and validate every file before writing any, back up, and refuse a bad one. Removal filters out only handlers containing `islet-hook.js`, so other tools' hooks survive.

How is CPU kept low?

Animation loops stop when nothing moves, helpers print only on change where possible, and wave data is limited to 30 per second. Transcripts are read from the tail only.

Security

What's the security model?

Page isolated from Node, fixed bridge API, server on loopback only, every external input validated, outside text set with `textContent`, and link schemes allowlisted.

What's the weakest point?

The hook server has no authentication, so any local program could fake events. They can't approve anything for real, because the answer goes back to whoever asked. I'd add a random token shared through the hook's config.

Could a malicious notification attack the app?

Its text is only ever shown as text. Clicking it opens a URL only if the scheme is on the allowlist; otherwise it opens the sending app.

Testing and process

How is it tested?

148 tests in 18 files using Node's built-in `node --test`, with no extra dependencies. Mostly logic: calendar rules, time zones, lyrics, hook install, and races. Fakes replace the network and `http.createServer`.

How do you test time zones?

The calendar tests run with the host set to different zones, such as Asia/Kolkata and UTC, including daylight-saving changes.

What isn't tested?

The real screen and mouse, and the PowerShell helpers. I check those by hand, and through Chrome DevTools using `--remote-debugging-port=9333`.

Did you use AI to build this?

Answer truthfully; the commit history publicly shows Claude and Codex as co-authors. A strong answer says what you decided, how you checked the result, and which bugs you found. This guide is there so you can back that up.

© 2026 Gopinath. All rights reserved.

SECTION 22

Cheat sheet and practice

The numbers to remember, then the exercises that make it stick.

Thing	Value	Thing	Value
Media check	every 1 s	Hook server	127.0.0.1:47321
Notification check	every 2 s	Permission hold	8 s default, 24 s max
Agent session check	every 2 s	Agent sessions shown	5, with last 3 actions
Screen redraw	every 500 ms	Session counts as recent	30 min
Wave data	about 30 per second	Hover open / close delay	120 ms / 400 ms
Download scan	every 1.5 s	Helper restart	after 2 s
Weather refresh	every 15 min	Window size	672 × 190 px
Source size	about 3,700 lines	Tests	148 in 18 files

Prove to yourself you get it

Close this guide and answer aloud. If you get stuck, go back to the section in brackets.

- Why can't the notch page read a file directly? (15, 18)
- Draw the path from clicking Next to the song changing. (17)
- What happens to a Claude permission request if Islet is closed? (17)
- What goes wrong if `saveConfig` writes straight to `config.json`? (18)

Change something: the fastest way to learn

Run `npm install` and `npm start`. Try these, one at a time. Undo each with `git checkout -- <file>`.

1. **Easy:** change hover-open from 120 to 1000 ms in `notch.js`. Restart, and feel why 120 was chosen.
2. **Easy:** start with `--remote-debugging-port=9333`, open the page from chrome://inspect, and call `setOpen(true)`.
3. **Medium:** make clipboard history keep 10 items, not 5. Find where the limit lives.
4. **Medium:** add a test to `test/lyrics.test.js` for a line with no words. Run `npm test`.
5. **Medium:** trace a new IPC message: add a channel in `preload.js`, send it from `notch.js`, and log it in `main.js`.
6. **Hard:** add a shared secret token to the hook server, in the hook and the bridge, plus a test that a wrong token gets a 400. This fixes a real limit, and makes a great interview story.

Islet

Designed and built by **Gopinath**

© 2026 Gopinath. All rights reserved. The source code is published under the MIT License (see LICENSE).
Spotify, Windows, Claude Code and Codex are trademarks of their owners.

© 2026 Gopinath. All rights reserved.